

Runique

Framework web Rust — Dossier projet personnel

Sébastien Alliot

Graduate Développeur Front-End — Reconversion

Présentation

- Cuisinier en reconversion vers le développement web
- 2 ans de pratique — Python/Django puis Rust
- Échec d'examen (déc. 2025) → période de démotivation → découverte de Rust
- Constat en reconstruisant un projet : les patterns se répètent
- **Décision : construire un framework plutôt que copier-coller**

Problème posé

L'écosystème Rust web (Axum, Actix) est puissant mais **bas niveau** :

- Chaque projet repart de zéro
- Pas de gestion native des formulaires, sessions, CSRF, admin, migrations
- Django avait résolu tout cela il y a 20 ans

L'objectif n'est pas de surpasser Django — mais de transposer sa philosophie "batteries included" dans l'écosystème Rust.

Architecture technique

Pattern découvert progressivement, pas conçu à l'avance :

Récupération → Traitement → Validation → Persistance

- S'est révélé universel : formulaires, requêtes HTTP, pipeline de sécurité
- Émergé de la répétition et de la simplification
- Le compilateur Rust valide naturellement ce qui est bien structuré

Rust et la programmation orientée objet

Rust n'a pas d'héritage de classes — mais une alternative :

POO classique	Rust
<code>class</code>	<code>struct</code> (données)
Héritage	<code>trait</code> (comportements)
Interface	<code>trait</code>

Runique exploite ce modèle :

- `RuniqueUser` , `RuniqueForm` , `AdminResource` → traits à implémenter
- **Composition** plutôt qu'héritage — extensible sans couplage fort

Fonctionnalité clé — Le système de slots

Problème : les middlewares Rust doivent être enregistrés dans un ordre précis.

Solution : chaque middleware a une priorité numérique fixe.

Slot	Middleware
5	Compression
50	Session
60	CSRF
70	Host validation

L'utilisateur déclare dans n'importe quel ordre → le framework garantit l'ordre correct.

Fonctionnalité — derive_form!

Un seul DSL génère l'entité SeaORM, la migration SQL et le formulaire :

```
derive_form! {  
  Article {  
    fields: {  
      titre: text [max_length: 200, required]  
      publie: bool [default: false]  
      auteur: fk(User) [required]  
      image: file [max_size: 2MB, allowed: image]  
    }  
    meta: { table_name: "articles", ordering: ["-created_at"] }  
  }  
}
```

Fonctionnalité — admin!{} ---

Une déclaration génère un panel CRUD complet :

```
admin! {  
  article: article::Model => ArticleForm {  
    title: "Articles",  
    list_display: [  
      ["titre", "Titre"],  
      ["auteur", "Auteur"],  
      ["publie", "Publié"],  
    ],  
    list_filter: [  
      ["publie", "Publié", 5],  
      ["auteur", "Auteur", 5],  
    ],  
  }  
}
```

Liste paginée, filtres, création, édition, suppression — sans une ligne de plus.

Fonctionnalité — Sécurité intégrée

Audit OWASP ZAP — 0 alerte critique

```
.middleware(|m| m
  .with_session_memory_limit(5 * 1024 * 1024, 10 * 1024 * 1024)
  .with_csp(|c| c
    .policy(SecurityPolicy::strict())
    .with_header_security(true)
    .with_upgrade_insecure(!is_debug())
  )
)
```

La validation des hôtes est conditionnée à `!is_debug()`. La redirection HTTPS est contrôlée par `ENFORCE_HTTPS=true` dans le `.env` — nécessite un reverse proxy qui transmet `X-Forwarded-Proto`. Si le proxy gère lui-même la redirection, laisser `ENFORCE_HTTPS=false` pour éviter une boucle.

Mise en production

runique.io — documentation + cours + démo du framework

- Déploiement simple : binaire autonome, VPS, ACME intégré (feature Cargo)
- Ou derrière Nginx qui termine le TLS
- Pas de dépendance runtime

Ce que la prod a révélé :

- Conflits de ports ACME sur multi-projets
- Bugs visibles uniquement en environnement réel

La production enseigne ce qu'on n'avait pas prévu de simuler.

Fiche technique

Langage	Rust — edition 2024
Version	2.1.12
HTTP	Axum 0.8 · ORM SeaORM 2.0.0-rc.38
Templates	Tera 1.20.1
Bases de données	PostgreSQL, MariaDB, SQLite
Hébergement	VPS — runique.io
Sécurité	Audit OWASP ZAP — 0 alerte critique
Documentation	Bilingue FR/EN — 88 pages
Cours	31 chapitres Rust publiés
Dépôt	github.com/seb-alliot/runique

Conclusion

Trois apprentissages sur moi-même :

1. **La patience** — un framework se construit par couches, pas en un sprint
2. **La persistance** — les bugs de prod n'apparaissent pas en local
3. **La vision long terme** — la motivation vient du chemin, pas de la ligne d'arrivée

Runique n'est pas terminé — il ne le sera probablement jamais complètement.
Mais c'est précisément ce qui le rend vivant.